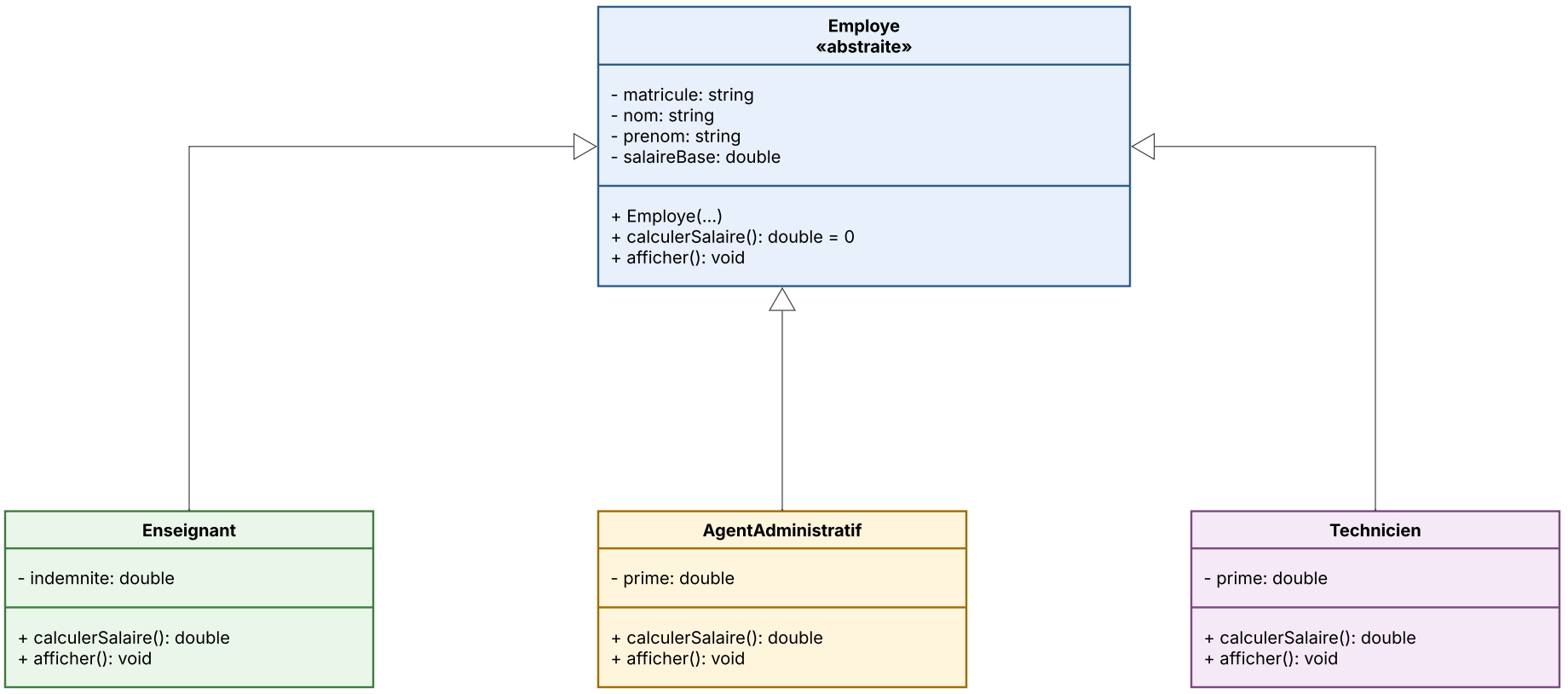



```cpp
#include <iostream>
#include <memory>
#include <string>
#include <vector>

class Employe {
protected:
    std::string matricule, nom, prenom;
    double salaireBase;

public:
    Employe(std::string m, std::string n, std::string p, double base)
        : matricule(std::move(m)), nom(std::move(n)),
          prenom(std::move(p)), salaireBase(base) {}
    virtual ~Employe() = default;

    virtual double calculerSalaire() const = 0;
    virtual void afficher() const {
        std::cout << matricule << " - " << prenom << " " << nom
                  << " | Salaire : " << calculerSalaire() << "\n";
    }
};

class Enseignant : public Employe {
private:
    double indemnite;
public:
    Enseignant(std::string m, std::string n, std::string p,
               double base, double ind)
        : Employe(std::move(m), std::move(n), std::move(p), base),
          indemnite(ind) {}

    double calculerSalaire() const override {
        return salaireBase + indemnite;
    }
};

class AgentAdministratif : public Employe {
private:
    double prime;
public:
    AgentAdministratif(std::string m, std::string n, std::string p,
                       double base, double pr)
        : Employe(std::move(m), std::move(n), std::move(p), base),
          prime(pr) {}

    double calculerSalaire() const override {
        return salaireBase + prime;
    }
};

class Technicien : public Employe {
private:
    double prime;
public:
    Technicien(std::string m, std::string n, std::string p,
               double base, double pr)
        : Employe(std::move(m), std::move(n), std::move(p), base),
          prime(pr) {}

    double calculerSalaire() const override {
        return salaireBase + prime;
    }
};

int main() {
    std::vector<std::unique_ptr<Employe>> personnel;
    personnel.push_back(std::make_unique<Enseignant>(
        "E001", "Dupont", "Alice", 3000, 750));
    personnel.push_back(std::make_unique<AgentAdministratif>(
        "A001", "Martin", "Bruno", 2200, 300));
    personnel.push_back(std::make_unique<Technicien>(
        "T001", "Bernard", "Claire", 2500, 450));

    for (const auto& employe : personnel) {
        employe->afficher(); // liaison dynamique
    }
    return 0;
}
```